Real-Estate Price Prediction: Decision Trees and Support Vector Machines

Leyla Aliyeva, Agil Asgarli, Mahammad Gurbanzade, Dilara Rzazade

MLE-AI-201 Machine Learning, AI Academy — Cohort II

***Abstract*—**We predict real-estate asking prices and classify listings into standard and premium tiers using bina.az data. Our pipeline cleans duplicate listings, handles missing values, encodes categorical variables, and prevents target leakage. We implement decision-tree models and a linear SVM trained with Pegasos, then compare them with scikit-learn baselines. After cleaning, 64,452 listings and 173 features remain. The premium threshold is the training-set median of 210,000 AZN. Our decision-tree classifier achieves 0.9180 test accuracy and 0.9588 ROC-AUC; the regression tree achieves an original-scale R-squared of 0.5144. The linear SVM reaches 0.8492 test accuracy and 0.9233 ROC-AUC, comparable to scikit-learn's linear SVMs but about seven points below the tree. These results demonstrate the usefulness of tree-based models for property listings, while the use of asking prices and a random split limits conclusions about future market prices. Code: `github.com/Dilara898/ML-Final_Project`.

***Index Terms*—**decision tree, support-vector machine, Pegasos, regression, classification, real-estate price prediction.

I. Introduction

Real-estate price estimation can help compare listings and understand how property attributes relate to asking prices. Prices depend on location, area, room count, and building characteristics, making this a useful prediction problem. This project addresses regression on log-transformed prices and classification into standard and premium tiers. Our contributions are a reproducible preprocessing pipeline, from-scratch NumPy decision-tree and Pegasos SVM implementations, comparison with scikit-learn baselines, and held-out test evaluation. Training-only preprocessing and explicit leakage prevention are used to make the evaluation more reliable.

II. Related Work / Background

Decision trees are supervised learning models that recursively partition the feature space. Classification trees commonly select splits by reducing impurity, measured using criteria such as Gini impurity or entropy. Regression trees instead seek splits that reduce squared prediction error. Their main advantage is the ability to represent nonlinear relationships and feature interactions without requiring a linear functional form. However, overly deep trees may overfit, so model complexity must be controlled.

Support Vector Machines (SVMs) seek a decision boundary with a large margin. The soft-margin formulation balances margin size against hinge-loss classification errors through a regularisation parameter. Pegasos optimises the regularised hinge-loss objective using stochastic subgradient updates, making linear SVM training practical on large datasets [1]. Random Fourier Features (RFF) approximate an RBF kernel through a randomized feature mapping, allowing a linear learner to represent nonlinear boundaries without constructing a full kernel matrix.

III. Data and Preprocessing

The dataset was collected from bina.az real-estate listings. After cleaning, 64,452 rows and 173 model input features remained. The data were divided into 45,116 training examples, 9,667 validation examples, and 9,669 test examples. The training set contains 49.37% premium listings. The premium threshold is 210,000 AZN, defined using the training-set median.

Two tasks were considered. Regression predicts the asking price (modelled as $\log(\text{price})$ because prices are heavy-tailed), while classification predicts whether a listing belongs to the premium tier.

The raw dump contained 100,775 rows and 51 columns. Repeated observations of the same listing were identified by listing URL (estate IDs were not unique per property); we kept the most recent scrape of each URL, which removed 36,321 rows and left 64,454. Two listings whose sale-price target was ambiguous were excluded after manual review, giving 64,452 rows. Both currency columns contained AZN throughout, so no conversion was needed. Area values recorded in *sot* were converted to square metres, and floor strings such as "7/9" were split into floor number and total floors. Missing renovation and mortgage information was encoded as "unknown" rather than assumed to mean "no". Remaining numerical gaps were filled with training-set medians within each property category, with missingness indicators for land area, floor, total floors, rooms and coordinates; such filled values are modelling placeholders, not verified measurements. The building-type column was dropped because about 99.85% of its values were missing.

The columns `unit_price` and `total_price` restate the target and were removed to prevent target leakage; identifiers, URLs, owner details and free text were not used. Model inputs come from an explicit list of 18 columns, which expand to 173 features after categorical encoding. Training rows (about 70%) were drawn at random with seed 42, the tier threshold was computed from the training median, and the remaining rows were split into validation and test sets stratified by tier. Scaling statistics come from the training split only. Upper limits for `area_m2` and `land_area_m2` are the 99th training percentile within each property category (learned only for categories with at least

Table 1. Dataset summary after cleaning.

Property	Value
Rows (after cleaning)	64,452
Features used	173
Train / Val / Test split	45,116 / 9,667 / 9,669
Premium share (train)	49.37%

100 observed training values); no rows were removed and prices were not capped. One listing whose coordinates conflicted with its stated location had them set to missing. No listing URL appears in more than one split. Different URLs may still describe the same physical property, so a random split can place near-duplicates in different splits.

Exploratory analysis of the cleaned training data (before capping and imputation) shows a long right tail in prices, so we model log(price); the plot alone does not establish a log-normal distribution. Area is positively associated with price, with wide variation within each property category, and some unusually low prices and large areas remain. Two- and three-room properties dominate the residential listings, and median price generally rises with room count, although area, location and renovation also differ between properties, so rooms alone do not explain the trend. The data are heavily concentrated in Baku listings, so the results mainly describe that market. The median threshold gives nearly balanced tiers: 22,840 standard and 22,276 premium listings in training (49.37% premium) and 49.45% premium in both the validation and test splits. Prices are advertised asking prices, not verified transaction prices.

IV. Method

A. Decision tree (from scratch)

The tree recursively selects feature splits that reduce node impurity for classification or squared prediction error for regression. Each example follows the learned rules to a leaf, which returns a class prediction or numerical estimate. Tree depth and leaf-size constraints control complexity. Models are evaluated on the held-out split and compared with scikit-learn decision-tree baselines.

B. Support vector machine (from scratch)

We implement a linear soft-margin SVM in NumPy and train it with Pegasos, a stochastic sub-gradient method on the regularised hinge-loss primal (no quadratic-programming solver). Labels are mapped to $\{-1,+1\}$ (premium $=+1$). At step t we draw one example i, set $\eta_t = 1/(\lambda t)$, and update

$$\mathbf{w} \leftarrow (1-\eta_t\lambda)\,\mathbf{w} + \eta_t\, y_i \mathbf{x}_i\, \mathbf{1}\Big[y_i\,\mathbf{w}^\top \mathbf{x}_i < 1\Big]. \qquad (1)$$

The bias is folded into $\mathbf{w}$ by appending a constant 1 to every feature vector, so the bias is also regularised (a difference from the standard formulation and from scikit-learn). Features are standardised with training-set statistics only. The margin width is $2/\|\mathbf{w}\|$ (computed on the weights without the bias), and support vectors are the training points with $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \le 1$. The objective J is logged during training. A constant learning-rate option is also implemented and compared below; all other reported results use the Pegasos schedule.

Table 2. Linear Pegasos, λ sweep (50k iterations $\approx$ 1.1 epochs).

λ	$\|w\|$	margin	SV share	train	val
10^{-5}	788.8	0.003	0.208	0.794	0.795
10^{-4}	79.9	0.025	0.222	0.801	0.801
10^{-3}	9.83	0.203	0.291	0.813	0.818
10^{-2}	2.37	0.845	0.408	0.838	0.844
10^{-1}	1.12	1.781	0.542	0.826	0.827
1	0.54	3.713	0.855	0.792	0.790

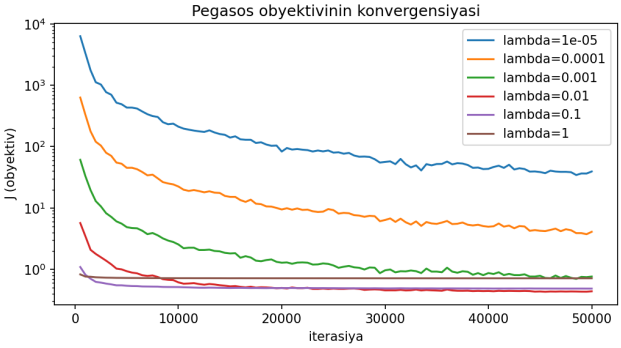


Figure 1. Pegasos objective J versus iteration for each λ (log scale). Small λ ($\le 10^{-3}$) is still decreasing after 50k iterations, whereas $\lambda \ge 10^{-2}$ is essentially flat.

For non-linear boundaries we use random Fourier features (RFF) rather than kernelised Pegasos: $z(\mathbf{x}) = \sqrt{2/D}\cos(W^\top \mathbf{x} + \mathbf{b})$ with $W_{jk} \sim \mathcal{N}(0, 2\gamma)$ and $b_k \sim U[0, 2\pi]$, fitted on the training split only. This approximates an RBF kernel, reuses the linear solver unchanged, and avoids an $n \times n$ kernel matrix.

C. SVM tuning and results

SVM hyperparameters were chosen on the validation split only; the test set was evaluated once, after the final configuration was fixed. We swept $\lambda \in \{10^{-5}, \ldots, 1\}$ (50,000 iterations), then, for RFF, $\gamma \in \{0.0005, \ldots, 0.03\}$ with $\lambda \in \{10^{-5}, 10^{-4}, 10^{-3}\}$ and $D = 300$. A follow-up checked longer training (50k, 200k, 500k iterations) and $D \in \{300, 1000\}$. All runs use seed 42. Baselines are scikit-learn's SGD classifier (hinge loss) and LinearSVC on standardised features.

Table 2 and Fig. 1 show the effect of λ: as λ grows, $\|\mathbf{w}\|$ shrinks and the margin widens, while validation accuracy peaks at $\lambda = 10^{-2}$ (with 50k iterations).

With longer training (500k iterations, about 11 epochs), $\lambda = 3 \times 10^{-3}$ gave the best validation accuracy (0.846) and was fixed as the final configuration. On the test set the SVM reaches accuracy 0.849, F1 0.850 and AUC 0.923; in 5-fold CV its F1 is 0.843 ± 0.002, versus 0.845 ± 0.006 for SGD-hinge and 0.839 ± 0.003 for LinearSVC, i.e. within one standard deviation of both baselines. The final model has $\|w\| = 2.52$, margin width 0.79, and 17,288 support vectors (38.3% of training data). Fitting takes about 6.4 s (SGD-hinge 1.1 s, LinearSVC 3.6 s) and prediction about 1 ms (Table 7).

Table 3. Learning-rate schedules at $\lambda = 3 \times 10^{-3}$, 500k iterations.

schedule	$\|w\|$	margin	SV share	train	val	final J
Pegasos	2.52	0.793	0.383	0.845	0.846	0.390
constant 0.001	2.54	0.788	0.380	0.843	0.844	0.395
constant 0.01	4.24	0.472	0.341	0.828	0.825	0.534
constant 0.1	15.1	0.132	0.264	0.787	0.782	1.867

Learning-rate schedule. At the final $\lambda = 3 \times 10^{-3}$ and 500k iterations, the Pegasos schedule $\eta_t = 1/(\lambda t)$ reached the lowest objective ($J = 0.390$) and the best validation accuracy (0.846). A small constant step ($\eta_0 = 0.001$) was almost as good ($J = 0.395$, 0.844), but larger constant steps degraded the solution: $\eta_0 = 0.01$ gave $\|w\| = 4.24$, margin 0.47 and validation accuracy 0.825, and $\eta_0 = 0.1$ gave $\|w\| = 15.1$, margin 0.13, $J = 1.87$ and accuracy 0.782 (Table 3). We therefore keep the Pegasos schedule.

RFF did not improve on the linear SVM: the best validation accuracy was 0.830 with $D = 300$ and 0.843 with $D = 1000$, versus 0.846 for the linear model. Increasing the number of iterations did not help at $D = 300$, whereas more components did, which suggests the feature budget, not training length, was limiting; we did not test this further. The decision tree outperforms the SVM by about seven accuracy points; a plausible reason (not verified) is that a linear model cannot capture interactions between, for example, location and area. Differences of about 0.004 in validation accuracy are within sampling noise for 9,667 validation examples, so close configurations cannot be ranked reliably. Remaining differences to scikit-learn come from the stochastic solver, the regularised bias, and the $C = 1/(n\lambda)$ correspondence being exact only at the optimum. Sweeps used a single seed.

V. Experimental Setup and Results

A. Protocol and metrics

Task A is evaluated with RMSE, MAE and R^2, both on log(price) (the training target) and on the original AZN scale. Task B (premium = 1) is evaluated with accuracy, precision, recall, F1 and ROC-AUC. All metrics are implemented in NumPy and unit-tested against scikit-learn. Final numbers use the test split (9,669 listings), on which models were fitted on the training split only. Uncertainty is the mean $\pm$ sample standard deviation over five folds of the training and validation rows; the folds are identical for our models and the baselines, stratified by tier, and the tier threshold and scaler are recomputed inside every fold.

Our classification tree uses entropy with depth 20 and `min_samples_leaf` = 5; our regression tree uses squared error with depth 20 and `min_samples_leaf` = 20; both use `min_samples_split` = 10. The baselines are scikit-learn decision trees with the same settings, an SGD classifier (hinge loss, $\alpha = \lambda$) and LinearSVC ($C = 1/(n\lambda)$), the last two with a `StandardScaler` inside a `Pipeline`. An RBF `SVC` was not run because its cost grows quickly with the 45,116 training rows. The whole benchmark is produced by `python -m src.run_all`; all seeds are fixed to 42, and the `-check-seeds` option confirmed that two complete runs produce identical test metrics.

Table 4. Task A on the test set. Price-scale errors are in AZN.

Model	RMSE	MAE	R^2	$\text{RMSE}_{\log}$	$R^2_{\log}$
Our tree	444,189	89,583	0.5144	0.3835	0.7740
sklearn tree	444,104	89,553	0.5146	0.3829	0.7746

Table 5. Task B on the test set (premium = 1).

Model	Acc.	Prec.	Rec.	F1	AUC
Our tree	0.9180	0.9168	0.9174	0.9171	0.9588
sklearn tree	0.9174	0.9158	0.9172	0.9165	0.9588
Our SVM	0.8492	0.8371	0.8630	0.8498	0.9233
SGD-hinge	0.8528	0.8392	0.8689	0.8538	0.9287
LinearSVC	0.8438	0.8260	0.8668	0.8459	0.9269

B. Comparison with scikit-learn

Table 4 and Table 5 give the test results. Our trees are practically indistinguishable from scikit-learn's: classification accuracy differs by 0.0006, ROC-AUC by less than 0.0001, and the log-scale RMSE by 0.0006. We did not isolate the cause of these small gaps; tie-breaking among equally good splits and the exact threshold convention are plausible sources. The regression tree explains 77.4% of the variance of log(price) but only 51.4% on the AZN scale, where a few very expensive listings dominate the squared error (the most expensive test listing costs 18.5 million AZN). For the same reason the price-scale RMSE is unstable across cross-validation folds (about $1.68 \times 10^6 \pm 2.28 \times 10^6$ AZN), so we compare regression models in cross-validation on the log scale: RMSE 0.390 ± 0.011 and R^2 0.767 ± 0.013 for both our tree and scikit-learn's.

The linear SVMs, ours included, are about seven accuracy points behind the trees, and the cross-validated differences between the three linear models are within about one standard deviation. Our implementations are slower than scikit-learn's (Table 7): our trees take roughly twenty to thirty times longer to fit, plausibly because scikit-learn's trees are compiled code whereas ours run in NumPy and Python; we did not profile this. The timings come from a single run, so they indicate magnitude only.

C. Error analysis

Fig. 7 groups test listings by true-price decile. The median absolute percentage error of the regression tree is between 7.8% and 11.5% for deciles D2–D8, rises to 12.0% in D9 and to 25.1% in D10, and is 22.6% in D1. The most expensive decile alone accounts for 63% of the total absolute price error, with a mean absolute error of about 553,000 AZN against 21,000–79,000 AZN in the other deciles, so the model is least reliable for luxury listings. The cheapest decile starts at 127 AZN, which suggests that some entries are data errors; we did not check this.

For classification, the tree makes 398 false positives and 395 false negatives on the test set (Fig. 3), and the SVM makes 803 and 655 (Fig. 8). The two models disagree on 15.4% of test listings; the tree is right and the SVM wrong on 1,075 of them, the reverse on 410.

The tree importances (Fig. 4) are dominated by floor area (0.48 for the classifier, 0.56 for the regressor), followed by

Table 6. Task B, 5-fold cross-validation on train+val (mean ± std).

Model	F1	ROC-AUC
Our tree	0.915 ± 0.004	0.955 ± 0.003
sklearn tree	0.915 ± 0.004	0.954 ± 0.003
Our SVM	0.843 ± 0.002	0.923 ± 0.004
SGD-hinge	0.845 ± 0.006	0.926 ± 0.004
LinearSVC	0.839 ± 0.003	0.925 ± 0.003

Table 7. Fit and prediction time on 45,116 training rows (single run).

Model	Fit (s)	Predict (ms)
Our tree (regression)	10.96	36.2
sklearn tree (regression)	0.50	4.9
Our tree (classification)	15.73	36.2
sklearn tree (classif.)	0.49	5.9
Our SVM (Pegasos)	6.41	1.1
SGD-hinge	1.09	10.9
LinearSVC	3.62	18.8

longitude and latitude and, further down, the land-plot category, land area and the new-building category. Impurity-based importances tend to favour continuous features with many split points, so they indicate how the fitted tree uses a feature rather than a causal effect. The SVM weights (Fig. 9) are largest for the number of rooms (1.69) and floor area (0.94), positive for the new-building category and negative for the old-building category and for several peripheral districts. Rooms and area are strongly related, so their individual weights should not be interpreted separately.

VI. Visual Results

The following figures provide complementary views of classification performance, feature relevance, and regression error.

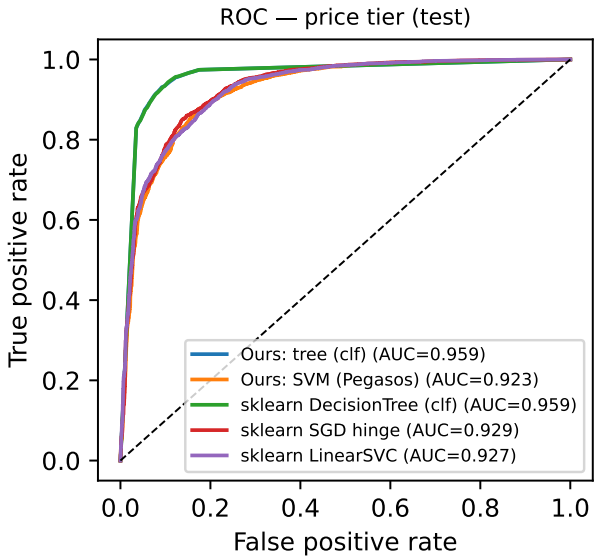


Figure 2. ROC curves comparing classification models. A larger AUC indicates better separation between the two classes across decision thresholds.

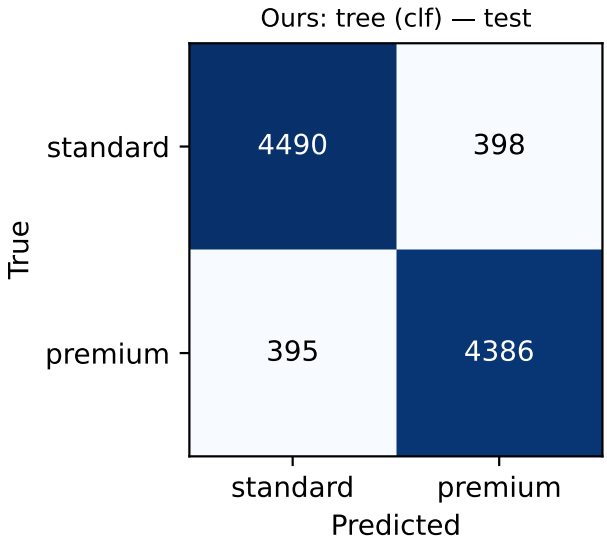


Figure 3. Confusion matrix for the decision-tree classifier. The diagonal contains correct predictions; off-diagonal entries represent errors.

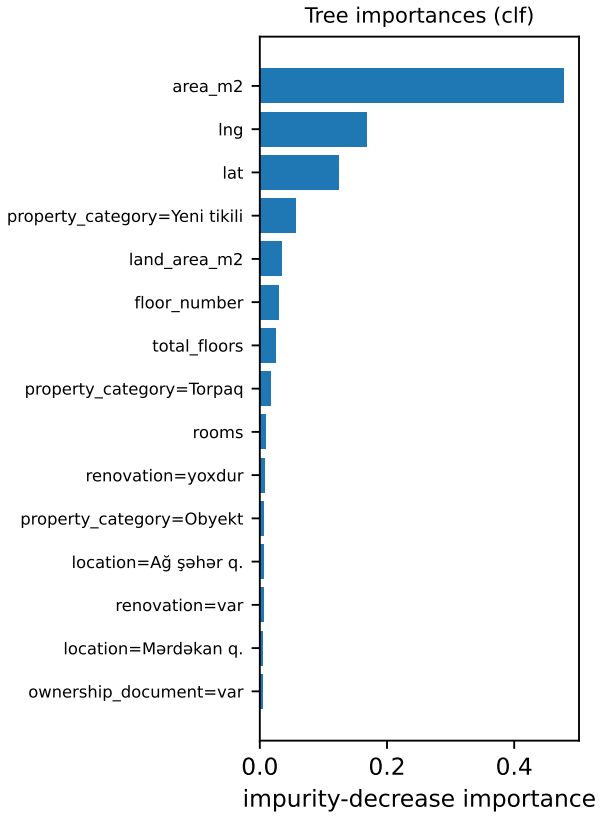


Figure 4. Feature importance for the decision-tree classifier. Importance scores describe the contribution of features to the fitted tree's splits.

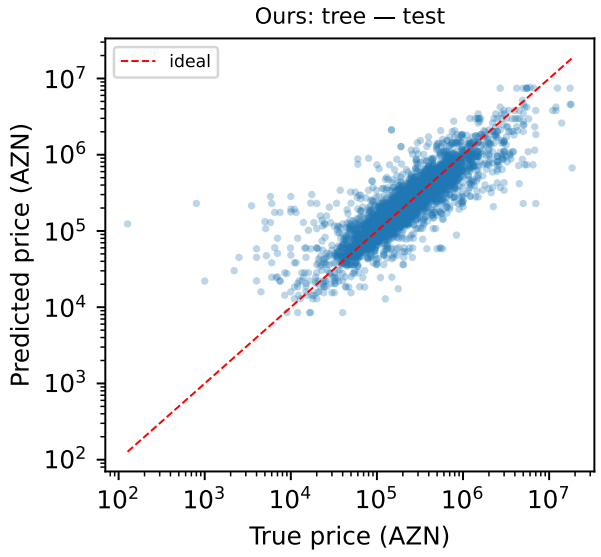


Figure 5. Predicted versus actual prices for the regression tree. Points closer to the ideal diagonal correspond to more accurate predictions.

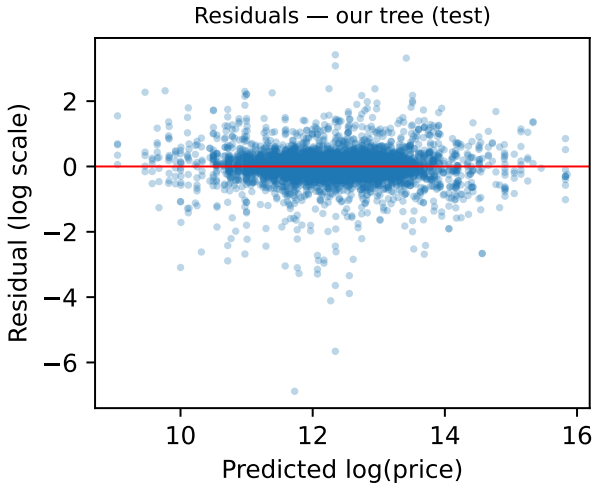


Figure 6. Regression residuals. Patterns in residuals can indicate systematic errors that a model does not capture.

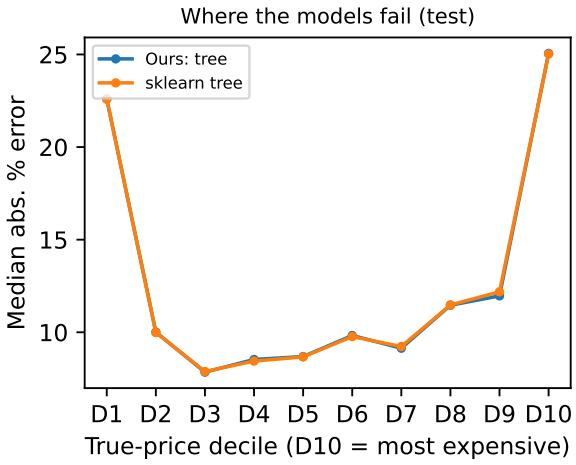


Figure 7. Prediction error across actual-price deciles, showing how errors vary between less expensive and more expensive properties.

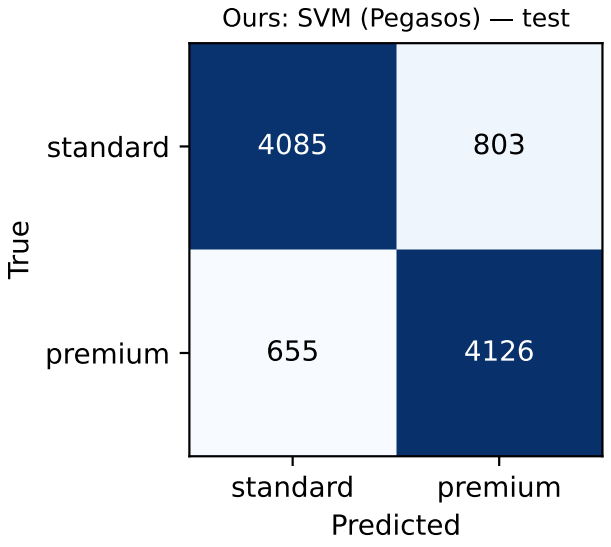


Figure 8. Confusion matrix for the SVM classifier.

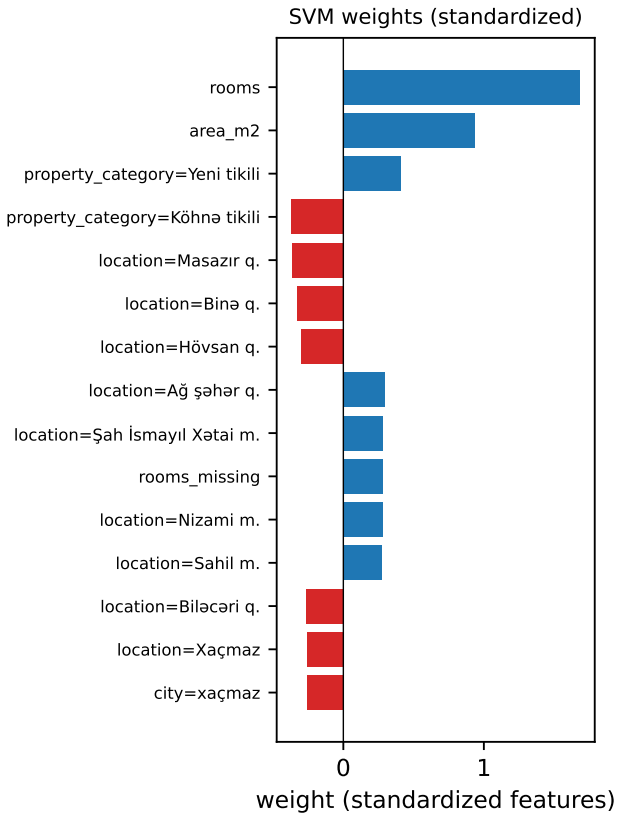


Figure 9. Learned SVM weights. Their magnitude indicates the model's linear coefficients, subject to the feature scaling used during training.

VII. Conclusion

The decision-tree classifier achieved 0.9180 test accuracy, an F1 score of 0.9171, and ROC-AUC of 0.9588. The from-scratch linear SVM reached 0.8492 test accuracy, comparable to scikit-learn's linear SVMs but about seven points below the tree; random Fourier features did not close the gap. The regression tree achieved an R-squared of 0.5144 on the original price scale, leaving substantial price variation unexplained. Trees can capture nonlinear relationships, while the linear SVM provides a useful contrasting baseline. Future work could add richer location features and test on a time-based split. Predictions concern asking prices, not confirmed sale prices.

AI-Assistance Disclosure

An AI assistant (Claude, Anthropic) was used in two parts of the project. For the SVM, it explained Pegasos and random Fourier features and drafted most of the code in `src/svm.py`, the sweep scripts, the SVM tests and the SVM sections of this report. For evaluation and integration, it drafted `src/evaluate.py` (metrics, ROC curve, k-fold splitting), the figure helpers, `src/run_all.py`, the associated tests, and the experimental-setup, results and error-analysis text of this report. Team members ran and checked this output: the metrics are tested against scikit-learn, and the pipeline was run end to end on the real dataset with a fixed seed. The decision tree and the SVM are our own NumPy implementations; scikit-learn is used only for baselines and as a reference in tests.

References

[1] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter, "Pegasos: primal estimated sub-gradient solver for SVM," *Mathematical Programming*, vol. 127, no. 1, pp. 3–30, 2011.